An intoduction to the **Auto-scrutineer**

Contents

- New submission folder structure
- Basic overview
  - Rule types
  - Explanation and examples

Part 1
Relevant to everyone

- In-depth overview (wip)
- Rule definitions (wip)
- VTK-pipelines (wip)

Part 2
Mostly relevant to staff and maintainers
(but everyone is welcome)

Disclaimer:
I am **NOT** a software engineer, so set your expectations accordingly.
Please let me know if there are critical bug/errors in my code, or other areas that might be improved.

Part 1

This part is relevant to everyone, competitors, staff etc.

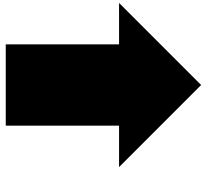
please familiarize yourself with the content.

New submission folder

Competitors must specify which .stl's are considered floor, frontwing etc.

Old structure

```
MyRacingTeam_Round1_v2
├── high_res_surfaces
│   ├── FW_endplate.stl
│   ├── FW_flap.stl
│   ├── rearwing.stl
│   └── etc
├── vehicle_body
│   ├── body.stl
│   ├── nose.stl
│   ├── mirror_strut.stl
│   └── etc
└── etc
    └── etc
```



New submission folder

```
MyRacingTeam_Round1_v2
├── frontwing_flaps
│   ├── FW_flap_1.stl
│   └── FW_flap_2.stl
├── frontwing_mainplane
│   └── fw_mp.stl
├── main_floor
│   └── floor_v13_revK_final2.stl
└── etc
    └── etc
```

In the new structure there is a folder for each "part" referenced in the rules.
Each folder can contain multiple stls.
So, more folders but no ambiguity about which stls belong to which parts.
Full list of parts should be specified in the rules.

After scrutineering, the .stl files are copied to a new "input_file" folder following the "old structure," making the submission "MFlow ready."

Rule types

Currently supported “rule types”:

- Obscure (read: to conceal etc.)
- Number of sections
- Top/bottom visibility
- Bounding box size (not currently in use)

Things like “10 mm rule” and “no floating parts” cannot be checked with this tool. If someone has ideas on how check these, please share them.

Obscure rule

The obscure rule is quite versatile. It can be used to test a range of different rules. These include:

- Part-X must be fully contained within RV-Y

- From *angle* Part-X must obscure RS-Y

- Part-Y must be lowest part of the car within *area*

Common for all these rules is that if X is visible from a certain angle, then that indicates a breach of the rule.

Two different geometries must be specified:

- “given_geometry” (X in the examples above)
- “ref_geometry” (Y in the example above)

An additional ”mask” geometry can also be specified. The mask can help to specify the *area* in the example above.

When checking this rule, a scene is created where the **given geometry** is colored red, and the **reference geometry** is colored black. Images of this scene are rendered. You can now check the rendered images for red pixels. Any red pixel = illegal geometry.

If an infringement is found, then the color palette is altered slightly, and an image is saved that more clearly shows the infringement (see example next slide)

To avoid Z-fighting issues, 6 copies of the reference geometry are added to the scene. Each translated 1 mm along –x, +x, -y, +y, -z and +z respectively

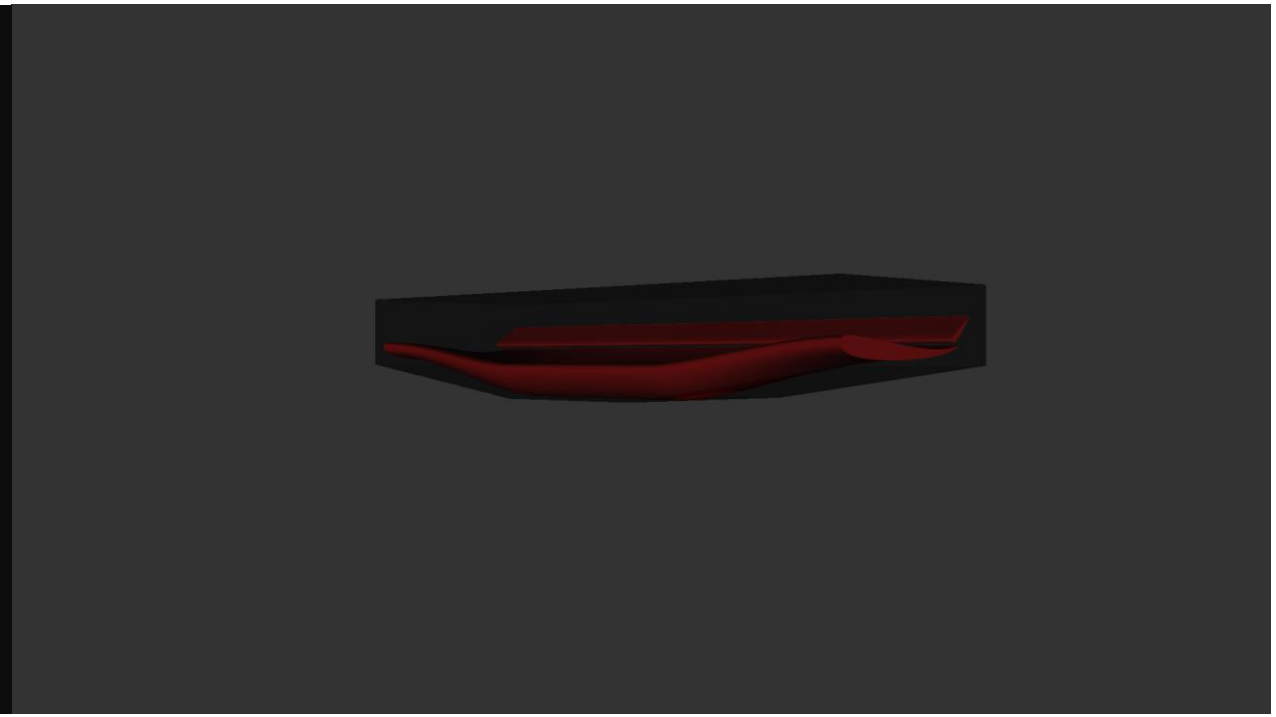
Obscure rule, example

What the rule-checker sees:
(flat-coloring)



Red = illegal

Rendered for more clarity:
(rendered with shadows and transparent reference)



The bottom surface of the mainplane is not fully contained within the RV

Number of sections

”Number of sections” –rule can be used to count number of sections in frontwings, rearwings etc.

It works by *cutting* the geometry with a number of planes. The planes are defined with a normal vector, and an origin point.

The normal vector is specified in the rules.

for example (0, 1, 0) for y-sections.

The origin points can be defined either as a strict ***list of specific*** points, or as a ***”semi-random”*** list of points in an interval. For semi-random sampling a ***”maximum unprobed interval”*** is given, this is the maximum distance between planes in the ***sampling_interval***.

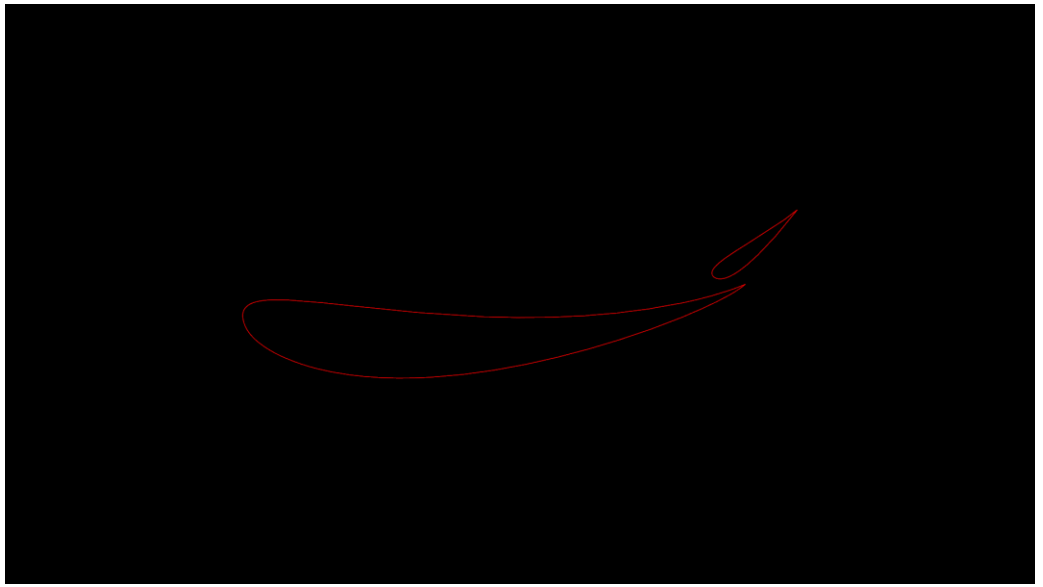
The 3d coordinates of the origin points are found by multiplying each scalar in the list with the normal vector of the plane.

An image of the cut section is rendered. Then the number of sections visible in the image are counted. (see next slide)

NB! Since it only checks a limited number of sections, small infringements may or may not be caught!

```
{
    'rule_section_name': '§2.0, mainplane sections',
    'rule_type': 'n sections',
    'angle': 'left side',
    'number_of_sections': "= 1",
    'given_geometry': 'rw_mainplane',
    'cutting_plane_normal' : [0, 1, 0],
    # 'sampling_method': 'specific',
    'sampling_method': 'semi_random',
    'sampling_interval': (0.0, 0.535),
    'max_unprobed_interval': 25e-3,
    # 'probing_locations': np.array([ 0.25 ]),
},
```

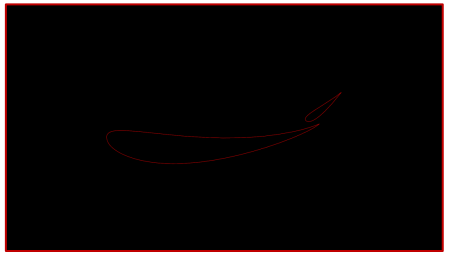
Rendered image of a section:



Number of sections

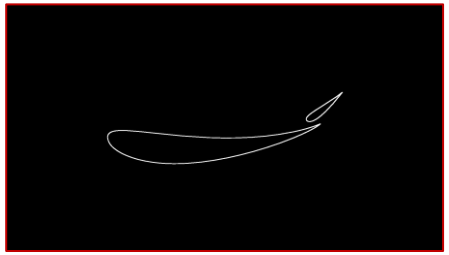
The function counts the number of sections in the rendered image, with the following method:

Rendered cut section
(red frame added for clarity)



(8bit RGBa array)

Convert to B/W

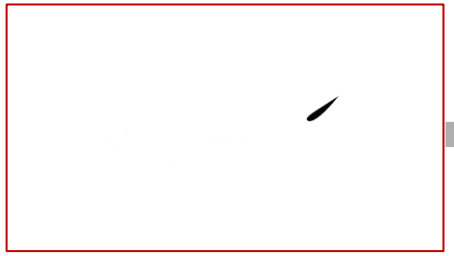


(Boolean array)

”Bucket fill” with white
from a corner



Find a black pixel,
Bucket fill from this pixel
Add one to the counter
Repeat until image is all white





Result: 2 sections

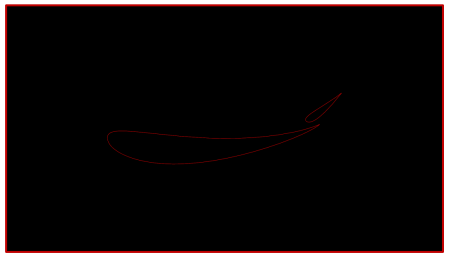
Why this way?
Another method could be to simply count the number of red sections. I decided against this approach because you often have overlapping geometries (because it is good for meshing). You don't want to count these overlapping regions as a section. This way you only count the number of sections in contact with the external flow. Imo this is closer to ideal.

Top/bottom visibility

Top/bottom visibility checks that every surface of the part is either visible from above or below. This rule takes sections of the given geometry, like the number of sections rule, but processes the images differently.

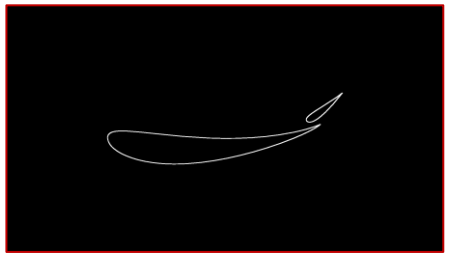
After ”bucket-filling” from a corner, it looks at every column of pixels in the image. Specifically, it counts the number of times the pixel value changes from black to white or white to black. If it counts more than 2 of these intersections then that suggest some overhang, and that means some surfaces must not be visible from above or below.

Rendered cut section
(red frame added for clarity)



(8bit RGBa array)

Convert to B/W



(Boolean array)

”Bucket fill” with white
from a corner



(this is a bad example, because top/bottom visibility doesn’t apply to the wings…)

Part 2

Mostly relevant to staff/contributors

This part should be mostly irrelevant to competitors that don’t wish to contribute to the script.

Rule definition

All rules are gathered in a single list. This list is defined in rules.py

Each rule is defined as a dictionary

The rule-checker iterates through the list of rules. The keyword: **”rule_type”** is important, this specifies which **“rule-function”** is called.

All rules must return a ***string*** that is added to the report.

New rules can be added by simply defining a new function and adding this function to the if-statement in the scrutineer function.

Two different rules in the rules list

```
{
    'rule_section_name': '§1.3',
    'rule_type': 'obscure',
    'angles': 'all',
    'ref_geometry': 'RV-RW-PYLON_v',
    'given_geometry': 'rw_pylon',
    'mask': 'none',
    'focus': 'given'
},

{
    'rule_section_name': '§2.0, mainplane sections',
    'rule_type': 'n sections',
    'angle': 'left side',
    'number_of_sections': "= 1",
    'given_geometry': 'rw_mainplane',
    'cutting_plane_normal' : [0, 1, 0],
    # 'sampling_method': 'specific',
    'sampling_method': 'semi_random',
    'sampling_interval': (0.0, 0.535),
    'max_unprobed_interval': 25e-3,
    # 'probing_locations': np.array([ 0.25 ]),
},
```

From scrutineer function:

```
# Check rules:
for rule in allRules:
    print('Checking rule: ' + rule['rule_section_name'] )

    if rule['rule_type'] == 'obscure':
        report += ruleObscure(rule, settings)

    elif rule['rule_type'] == 'n sections':
        report += ruleNSections(rule, settings)

    elif rule['rule_type'] == 'top bottom visibility':
        report += ruleTopBottomVisibility(rule, settings)
```

Obscure rule definition

- **Rule section name** is used for the report
- **Angles**, specifies the camera angles. Options include ”all”, ”above”, ”below” etc. See **camera_funs.py** for more info
- **Ref_geometry is** colored black
- **Given geometry is** colored red
- Focus specifies the part to focus the camera on, here the given geometry.

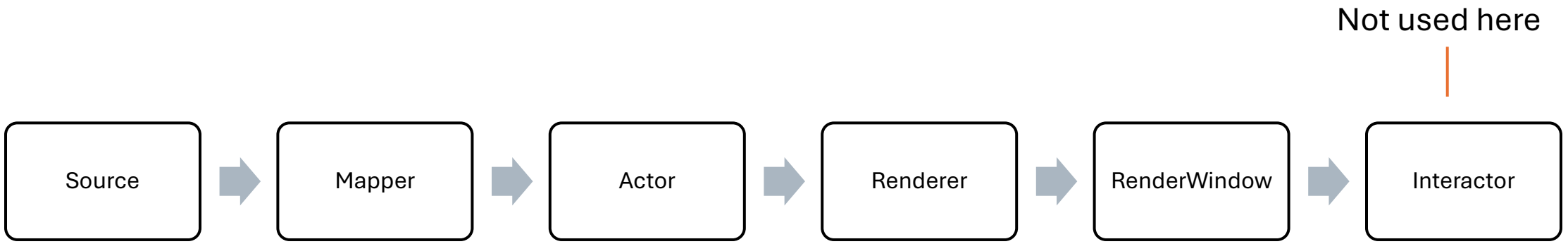
```
# FWEP
{
    'rule_section_name': '10.4 - FWEP inside RV-FW-Endplates',
    'rule_type': 'obscure',
    'angles': 'all',
    'ref_geometry': 'RV_FW_Endplates_V',
    'given_geometry': 'fw_ep',
    'focus': 'given'
},
```

Geometries can be **combined** by supplying a **list of parts** instead of a string. This functionality replaces the ”masks” used in previous versions, because this is better

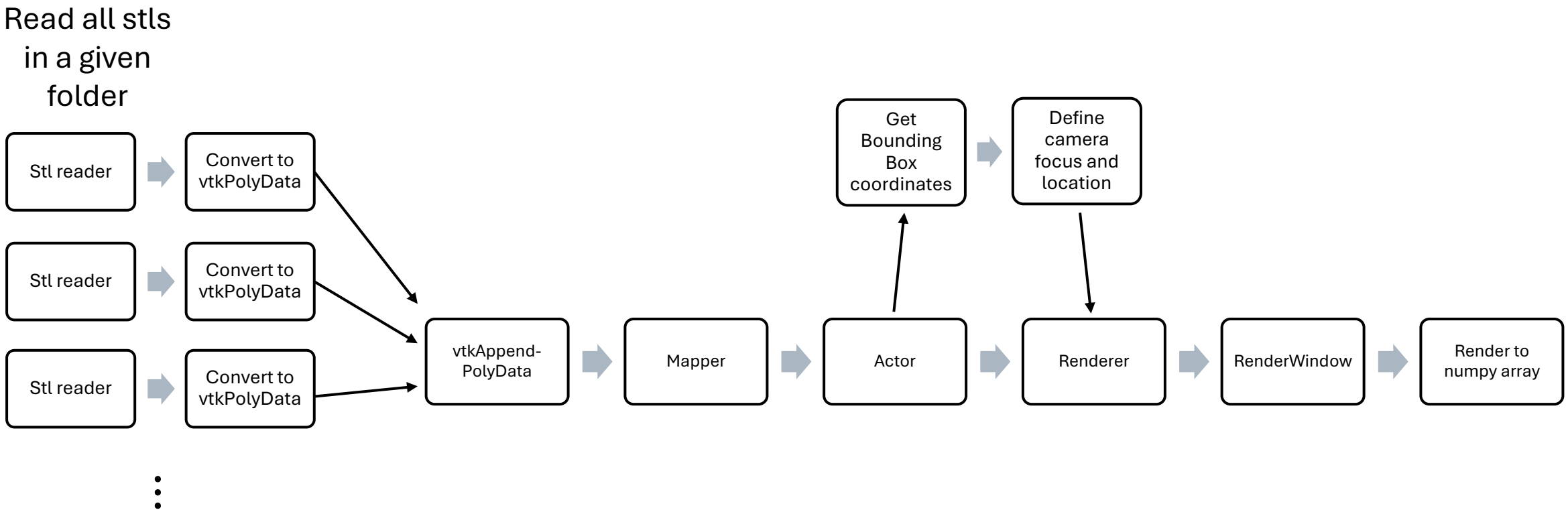
```
{
    'rule_section_name': '10.3 - Obscure RS-FW-Profiles',
    'rule_type': 'obscure',
    'angles': 'top',
    'ref_geometry': ['fw_mainplane', 'fw_flaps'], # Combine multiple geometries this way
    'given_geometry': 'RS_FW_PROFILES_V',
    'focus': 'given'
},
```

Overview of generic VTK-pipeline

(for reference)



Overview of VTK-pipeline for Obscure rule



Bounding box coordinates are now (march 2025) found from the actor

Overview of VTK-pipeline for N sections rule and top/bottom visibility rule

Most likely room for improvement.

